Materia:	Programación II		
Nivel:	2º Cuatrimestre		
Tipo de Examen:	Recuperatorio de Segundo Parcial		
Apellido(1):		Fecha:	5 dic 2025
Nombre/s(1):		Docente a cargo(2):	Baus / Bustos
División(1):	322-2	Nota(2):	
DNI(1):		Firma(2):	

(1) Campos a completar solo por el estudiante en caso de imprimir este enunciado en papel.
(2) Campos a completar solo por el docente en caso de imprimir este enunciado en papel.

Sistema de Gestión de Robots Exploradores de Marte

La Agencia Espacial Internacional está desarrollando una misión de exploración en Marte utilizando **robots autónomos**.
Cada robot cumple tareas como recolección de muestras, medición de parámetros ambientales, mapeo topográfico y asistencia a módulos habitacionales.

El objetivo del examen es implementar un sistema que permita **registrar, ordenar, buscar, transformar y persistir información** sobre estos robots exploradores, utilizando POO, genéricos, interfaces funcionales y archivos.
No se deben usar Streams.

1. Interface Almacenable<T>

(Igual estructura que en exámenes anteriores, pero este examen la define completamente)

Crear una interfaz genérica que permita almacenar y operar con elementos de tipo `T`.

Debe incluir:

Operaciones básicas

- `void agregar(T elemento);`
- `void eliminarSegun(Predicate<T> criterio);`
- `List<T> obtenerTodos();`
- `T buscar(Predicate<T> criterio);`

Ordenamientos

- `void ordenar();`
- `void ordenar(Comparator<T> comparador);`

Procesamiento

- `List<T> filtrar(Predicate<T> criterio);`
- `List<T> transformar(Function<T,T> operador);`
- `int contar(Predicate<T> criterio);`

Persistencia

- `void guardarEnBinario(String ruta) throws Exception;`
- `void cargarDesdeBinario(String ruta) throws Exception;`
- `void guardarEnCSV(String ruta) throws Exception;`
- `void cargarDesdeCSV(String ruta, Function<String,T> fromCSV) throws Exception;`
- `void guardarEnJSON(String ruta) throws Exception;`

---

2. Enum TipoRobot

Los robots pueden clasificarse en:

`RECOLECTOR, SENSORIAL, GEOLOGICO, ASISTENCIA, TOPOGRAFICO`

---

3. Clase RobotMarte

Cada robot posee:

- **id** (int): identificador único
- **nombre** (String): nombre operativo
- **tipo** (TipoRobot)
- **nivelBateria** (int): 0 a 100
- **anioFabricacion** (int): año de construcción para la misión
- **kmRecorridos** (double): distancia total recorrida en Marte

La clase debe:

- Implementar `CSVConvertible`
- Implementar `Comparable<RobotMarte>`
- Tener constructor completo + getters
- Implementar:
  - `toCSV()`
  - `static String toHeaderCSV()`
  - `static RobotMarte fromCSV(String linea)`
  - `toString()`
- **Orden natural requerido:**
  1. Mayor nivel de batería primero
  2. A igual nivel, más kilómetros recorridos primero

---

4. Clase genérica Inventario<T>

Debe:

- Implementar `Almacenable<T>`
- Utilizar `List<T>`
- **NO usar Streams**
- Implementar:
  - filtros
  - transformaciones
  - conteo
  - ordenamientos
  - persistencias:
    - BINARIO
    - CSV (con encabezado)
    - JSON pretty

---

5. Programa Principal (Tester)

El método `main` debe ejecutar exactamente en este orden:

1. Crear un inventario de `RobotMarte`.
2. Agregar **al menos seis robots** con distintos tipos, baterías y km recorridos.
3. Mostrar el inventario original.
4. Aplicar el **orden natural** (batería desc., km desc.).
5. Ordenar por **nombre alfabético** → completar Comparator.
6. Filtrar robots con **nivelBateria < 30** → completar Predicate.
7. Transformar robots de tipo **RECOLECTOR** aumentando su kmRecorridos en un **20%** → completar Function.
8. Contar robots fabricados **antes de 2018** → completar Predicate.
9. Guardar el inventario en:
   - `data/robots.bin`
   - `data/robots.csv`
   - `data/robots.json`
10. Crear un nuevo inventario y cargar los robots desde CSV usando `fromCSV`.
11. Mostrar el inventario cargado desde CSV.

```
public static void main(String[] args) {

    try {
        Almacenable<RobotMarte> inv = new Inventario<>();

        inv.agregar(new RobotMarte(101, "Ares-1", TipoRobot.RECOLECTOR, 85, 2019,
123.5));
        inv.agregar(new RobotMarte(102, "Vulcan-7", TipoRobot.SENSORIAL, 40, 2020,
88.2));
        inv.agregar(new RobotMarte(103, "GeoMax", TipoRobot.GEOLOGICO, 25, 2017,
210.4));
        inv.agregar(new RobotMarte(104, "TopoTrack", TipoRobot.TOPOGRAFICO, 77, 2021,
156.0));
        inv.agregar(new RobotMarte(105, "HelperX", TipoRobot.ASISTENCIA, 50, 2018,
95.3));
        inv.agregar(new RobotMarte(106, "Ares-2", TipoRobot.RECOLECTOR, 33, 2016,
178.9));

        System.out.println("=== Inventario Original ===");
```

```java
        for (RobotMarte r : inv.obtenerTodos()) {
            System.out.println(r);
        }

        // 1) Orden natural
        inv.ordenar();
        System.out.println("\n=== Orden Natural ===");
        for (RobotMarte r : inv.obtenerTodos()) {
            System.out.println(r);
        }

        // 2) Ordenar por nombre → completar Comparator
        inv.ordenar(/* Comparator por nombre */);
        System.out.println("\n=== Ordenados por Nombre ===");
        for (RobotMarte r : inv.obtenerTodos()) {
            System.out.println(r);
        }

        // 3) Filtrar robots con batería < 30 → completar Predicate
        System.out.println("\n=== Robots con Batería < 30% ===");
        List<RobotMarte> criticos =
            inv.filtrar(/* Predicate */);
        for (RobotMarte r : criticos) {
            System.out.println(r);
        }

        // 4) Transformar RECOLECTOR → +20% km → completar Function
        System.out.println("\n=== Transformación RECOLECTORES ===");
        List<RobotMarte> transformados =
            inv.transformar(/* Function */);
        for (RobotMarte r : transformados) {
            System.out.println(r);
        }

        // 5) Contar robots fabricados antes de 2018 → completar Predicate
        int antiguos = inv.contar(/* Predicate */);
        System.out.println("\nRobots fabricados antes de 2018: " + antiguos);

        // Persistencias
        inv.guardarEnBinario("src/data/robots.bin");
        inv.guardarEnCSV("src/data/robots.csv");
        inv.guardarEnJSON("src/data/robots.json");

        // Carga desde CSV
        Almacenable<RobotMarte> invCSV = new Inventario<>();
        invCSV.cargarDesdeCSV("src/data/robots.csv",
                              /* Function fromCSV */);

        System.out.println("\n=== Inventario cargado desde CSV ===");
        for (RobotMarte r : invCSV.obtenerTodos()) {
            System.out.println(r);
        }

    } catch (Exception e) {
        System.err.println(e.getMessage());
    }
}
```